```cpp
// Shrey Deogade & Aadesh Jay Harikumar
// Period 4
// January 9th, 2025
// Midterm Project

#include <iostream>
#include <stdlib.h>
#include <string>
#include <time.h>
using namespace std;

// Size of the chessBoard
const int SIZE = 8;

// Enum to enumerate all the possible states of a Square (either taken up by a
// Queen, or Empty)
enum SquareStatus
{
    Empty,
    Queen,
};

// The function prototypes
void solveEightQueens(SquareStatus[SIZE][SIZE]);
void initializeEmptyMatrix(SquareStatus[SIZE][SIZE]);
void printChessboard(SquareStatus[SIZE][SIZE]);

bool checkRow(SquareStatus[SIZE][SIZE], int);
bool checkColumn(SquareStatus[SIZE][SIZE], int);
bool checkDiagonals(SquareStatus[SIZE][SIZE], int, int);

bool columnTraversal(SquareStatus[SIZE][SIZE], int);

int main()
{
```

```
36      // Seed srand at time(0) for a pseudorandom number generator
37      srand(time(0));
38
39      // Initialize an empty chessBoard matrix
40      SquareStatus chessBoard[SIZE][SIZE] = {};
41
42      // Solve Eight Queens using the previous chessBoard matrix
43      solveEightQueens(chessBoard);
44
45      // Print the final result
46      printChessboard(chessBoard);
47      return 0;
48  }
49
50  void printChessboard(SquareStatus chessBoard[SIZE][SIZE])
51  {
52      // The ANSI escape codes for colors
53      string WHITE = "\033[0;107m";
54      string GRAY = "\033[0;42m";
55
56      string QUEEN_ON_WHITE = "\033[0;34;107m";
57      string QUEEN_ON_GRAY = "\033[0;34;42m";
58      string RESET = "\033[0m";
59
60      // Check if should print white; else gray
61      bool colorWhite = false;
62
63      cout << "   A  B  C  D  E  F  G  H" << endl;
64
65      // Iterate through every element in the matrix
66      for (int i = 0; i < SIZE; i++)
67      {
68          cout << SIZE - i << " ";
69
70          for (int j = 0; j < SIZE; j++)
```

```
71          {
72              // Make a temporary square variable to house the current chessBoard piece
73              SquareStatus square = chessBoard[i][j];
74
75              // Choose which color to print
76              string currentColor = (colorWhite) ? WHITE : GRAY;
77              string GOLD = (colorWhite) ? QUEEN_ON_WHITE : QUEEN_ON_GRAY;
78
79              // Make the symbol either a space (" ") or a Q ("Q") depending if the
80              // square is a queen or not
81              string symbol = (square == Queen) ? GOLD + "Q " + RESET : "  ";
82
83              // Construct and cout a string that concatenates all the previous stuff
84              cout << currentColor << " " << symbol << currentColor << "" << RESET;
85
86              // Make white its inverse to alternate colors between squares
87              colorWhite = !colorWhite;
88          }
89
90          // Make white its inverse to alternate colors between rows
91          colorWhite = !colorWhite;
92          // Newline
93          cout << " " << SIZE-i << endl;
94      }
95
96      cout << "   A  B  C  D  E  F  G  H" << endl;
97  }
98
99  void solveEightQueens(SquareStatus chessBoard[SIZE][SIZE])
100 {
101     // The main interface to solveEightQueens on a given chessBoard.
102
103     do
104     {
105         // Make sure the chessBoard is empty
```

```
106         initializeEmptyMatrix(chessBoard);
107
108         // Start off at a random position
109         int startingPosition = rand() % 8;
110         chessBoard[startingPosition][0] = Queen;
111         // And solve until the problem is solved
112     } while (!columnTraversal(chessBoard, 1));
113 }
114
115 void initializeEmptyMatrix(SquareStatus chessBoard[SIZE][SIZE])
116 {
117     // Initialize an empty matrix by iterating through every element and setting
118     // it to 0
119     for (int i = 0; i < SIZE; i++)
120     {
121         for (int j = 0; j < SIZE; j++)
122         {
123             chessBoard[i][j] = Empty;
124         }
125     }
126 }
127
128 bool columnTraversal(SquareStatus chessBoard[SIZE][SIZE], int column)
129 {
130     // Recursively traverse through the columns to solve the board - else false is
131     // handled by the
132     //   solveEightQueens function
133
134     // Base case - if the column is greater than or equal than the size, just
135     // return true
136     if (column >= SIZE)
137     {
138         return true;
139     }
140
```

```
141     // Iterate through every element of the column
142     for (int i = 0; i < SIZE; i++)
143     {
144         // Place a queen at the current (i, column) and later remove if it doesn't work
145         chessBoard[i][column] = Queen;
146 
147         // Store a bool to see if all the rows, columns, and diagonals' requirements
148         // are fulfilled
149         bool requirementsFulfilled = checkRow(chessBoard, i) &&
150             checkColumn(chessBoard, column) &&
151             checkDiagonals(chessBoard, i, column);
152 
153         // Execute only if the previous bool is true and if the recursive call for
154         // the next column is also true
155         if (requirementsFulfilled && columnTraversal(chessBoard, column + 1))
156         {
157             return true;
158         }
159 
160         // If the current queen doesn't work, make it Empty (failed so many times
161         // because I forgot to do this)
162         chessBoard[i][column] = Empty;
163     }
164     // If no queens can be placed in the column, return false to backtrack
165     return false;
166 }
167 
168 bool checkRow(SquareStatus chessBoard[SIZE][SIZE], int row)
169 {
170     // Check if a row has any queens, and if the number of queens is not 1,
171     // return false
172 
173     // Accumulator to keep track of how many queens there are in a row
174     int sum = 0;
175 
```

```cpp
    // Iterate through all elements in the row
    for (int i = 0; i < SIZE; i++)
    {
        // Increment sum if a Queen is found
        if (chessBoard[row][i] == Queen)
        {
            sum++;
        }
    }

    // There should be only 1 queen per row, so if it doesn't, it's false
    // because it doesn't fulfill the "1 queen per row" requirement
    return sum == 1;
}

bool checkColumn(SquareStatus chessBoard[SIZE][SIZE], int column)
{
    // Check if a column has any queens, and if the number of queens is not 1,
    // return false

    // Acummulator to keep track of how many queens there are in a column
    int sum = 0;

    // Iterate through all elements in the column
    for (int i = 0; i < SIZE; i++)
    {
        // Increment sum if a Queen is found
        if (chessBoard[i][column] == Queen)
            sum++;
    }

    // There should be only 1 queen per column, so if it doesn't, it's false
    // because it doesn't fulfill the "1 queen per column" requirement
    return sum == 1;
}
```

```
211
212 bool checkDiagonals(SquareStatus chessBoard[SIZE][SIZE], int row, int column)
213 {
214     // There are 2 types of diagonals:
215     // * the ones that go from top-left to bottom-right
216     // * the ones that go from bottom-left to top-right
217
218     // Diagonal type 1:
219     // Iterate through each element and see if there are any queens on a diagonal
220     // twice
221     for (int i = 0; i < SIZE; i++)
222     {
223         for (int j = 0; j < SIZE; j++)
224         {
225             // Store the current square to make it easier to access
226             SquareStatus square = chessBoard[i][j];
227
228             // i == row and j == column checks to see if you accidently go over a
229             // diagonal where a queen is already placed
230             if (i == row && j == column)
231             {
232                 continue;
233             }
234
235             // If 2 points are diagonally aligned, the same variables should subtract
236             // and get the same number
237             // Based on slope formula and y=mx+b
238             bool isDiagonal = i - row == j - column;
239
240             // See if the current square is a queen and is a diagonal. If it is, then
241             // return false because it doesn't fulfill the "1 queen per diagonal"
242             // requirement Uses previously assigned isDiagonal variable
243
244             if (square == Queen && isDiagonal)
245             {
```

```
246                     return false;
247                 }
248             }
249         }
250
251         // Diagonal type 2 (mostly the same thing as the first diagonal):
252         // Iterate through each element and see if there are any queens on a diagonal
253         // twice
254         for (int i = 0; i < SIZE; i++)
255         {
256             for (int j = 0; j < SIZE; j++)
257             {
258                 // Store the current square to make it easier to access
259                 SquareStatus square = chessBoard[i][j];
260
261                 // i == row and j == column checks to see if you accidently go over a
262                 // diagonal where a queen is already placed
263                 if (i == row && j == column) {
264                     continue;
265                 }
266
267                 // If 2 points are diagonally aligned, the same variables should subtract
268                 // and get the same number, except you switch the numbers in one of the
269                 // things because it's the reverse of the 1st type of diagonal
270                 // Based on slope formula and y = mx+b
271                 bool isDiagonal = i - row == column - j;
272
273                 // See if the current square is a queen and is a diagonal. If it is, then
274                 // return false because it doesn't fulfill the "1 queen per diagonal"
275                 // requirement
276                 //
277                 // Uses previously assigned isDiagonal variable
278                 if (square == Queen && isDiagonal)
279 {
280                     return false;
```

```
281             }
282         }
283     }
284
285     // Return true if there isn't another queen in the same diagonal as the
286     // original one (row, column)
287     return true;
288 }
```